Lecture 23 (B22): Gaussian Process Regression

Kernels, posterior, hyperparameter learning, deep kernel learning

Simon Scheidegger

HEC, University of Lausanne

Why Gaussian Processes?

- **Nonparametric:** No fixed architecture; the model grows with data.
- **Built-in uncertainty quantification:** Posterior mean *and* variance at every point.
- **Principled kernel selection:** Encode prior beliefs about smoothness, periodicity, etc.
- **Hyperparameter learning:** Marginal likelihood provides a principled objective.

Complementary to DNNs

- GPs: best for moderate dimensions ($d \leq 10$) with built-in UQ.
- DNNs: best for high dimensions ($d \gg 10$) with large data.
- Combining both is a powerful strategy (cf. slide 9).

Foundations: Rasmussen & Williams (2006); MacKay (1992); Krause, Singh & Guestrin (2008). **In DP:** Engel, Mannor & Meir (2005); Deisenroth, Rasmussen & Peters (2009). **Sparse / scalable:** Titsias (2009); Hensman, Fusi & Lawrence (2013).

Recall: Multivariate Gaussians

A random vector $\boldsymbol{x} \in \mathbb{R}^d$ is multivariate Gaussian if

$$\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$$

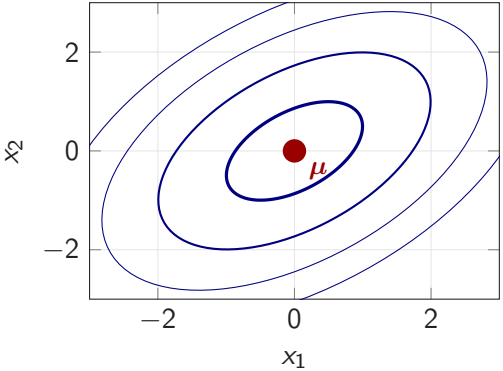
with density

$$p(\boldsymbol{x}) \propto \exp\left(-\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right)$$

Key fact

A multivariate Gaussian is fully characterized by its mean vector $\boldsymbol{\mu}$ and covariance matrix Σ.

2D Gaussian contours ($\rho = 0.5$)



Joint and Conditional Distributions

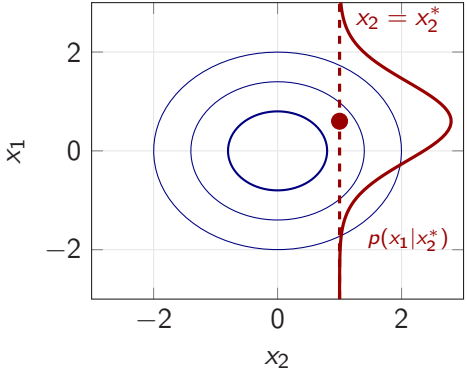
Joint distribution:

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix} \right)$$

Conditional distribution:

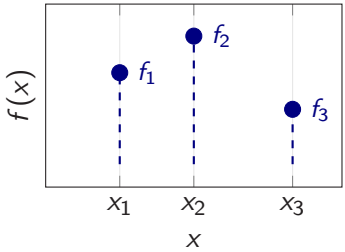
$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$$
$$\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

This is the mathematical engine behind GP regression.



"Slicing" the joint at $x_2 = x_2^*$ yields a 1D Gaussian.

From Observations to Interpolation



Assume function values are jointly Gaussian:

$$\begin{pmatrix} f_1 \\ f_2 \\ f_3 \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix}\right)$$

Nearby points (f_1, f_2) should be more correlated than distant ones (f_1, f_3).

Example: $K = \begin{pmatrix} 1.0 & 0.7 & 0.2 \\ 0.7 & 1.0 & 0.6 \\ 0.2 & 0.6 & 1.0 \end{pmatrix}$

Covariance via a kernel function: $K_{ij} = k(x_i, x_j)$

$\Rightarrow$ The kernel encodes our prior belief about function smoothness.

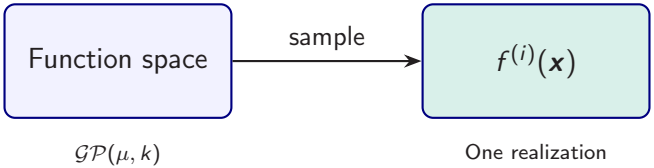
GP: A Distribution Over Functions

Definition

A Gaussian Process is a collection of random variables, any finite number of which have a joint Gaussian distribution:

$$f(\boldsymbol{x}) \sim \mathcal{GP}\big(\mu(\boldsymbol{x}),\ k(\boldsymbol{x}, \boldsymbol{x}')\big).$$

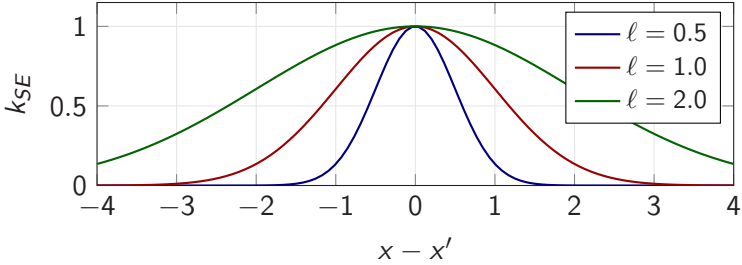
- **Mean function:** $\mu(\boldsymbol{x}) = \mathbb{E}[f(\boldsymbol{x})]$ (often set to 0).
- **Covariance function (kernel):** $k(\boldsymbol{x}, \boldsymbol{x}') = \mathbb{E}\big[(f(\boldsymbol{x}) - \mu(\boldsymbol{x}))(f(\boldsymbol{x}') - \mu(\boldsymbol{x}'))^\top\big]$.



The Squared Exponential (RBF) Kernel

$$k_{SE}(x, x') = \sigma_f^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

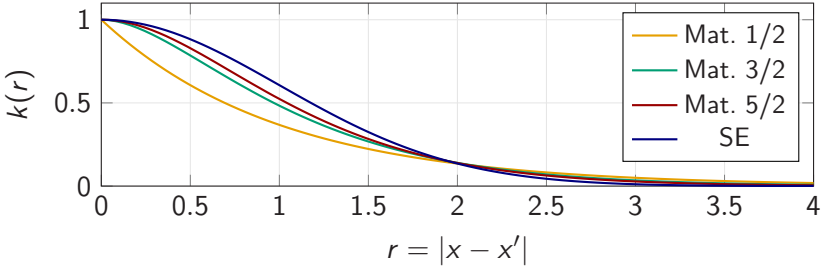
- ℓ = **length scale** (horizontal correlation range)
- σ_f^2 = **signal variance** (vertical scale)



Larger $\ell \Rightarrow$ smoother functions; smaller $\ell \Rightarrow$ more wiggly.

The Matérn Kernel Family

$$k_{\text{Mat}}(r) = \sigma^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}\, r}{\ell}\right)^{\nu} K_{\nu}\left(\frac{\sqrt{2\nu}\, r}{\ell}\right), \quad r = |x - x'|$$

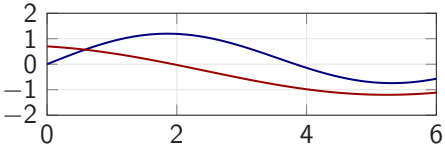


- ν controls smoothness: $\nu = 3/2 \Rightarrow$ once differentiable; $\nu = 5/2 \Rightarrow$ twice; SE $\Rightarrow$ ∞-smooth.
- **Use case:** Matérn when the true function is *not* infinitely smooth (e.g., has kinks).

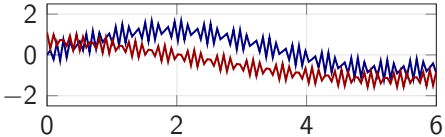
Combining Kernels

Kernels can be added and multiplied to compose complex priors from simple blocks.

SE kernel (smooth trend)



SE + Periodic (trend + seasonality)



Kernel algebra

- $k_1 + k_2$ (OR):
  high if *either* is high
  $\Rightarrow$ trend + seasonality
- $k_1 \times k_2$ (AND):
  high only if *both* high
  $\Rightarrow$ locally periodic

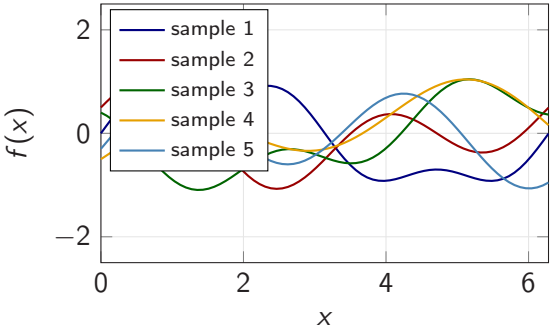
Compose complex priors from simple building blocks.

Sampling from the GP Prior

Algorithm

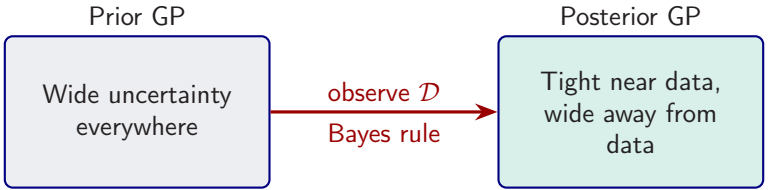
1. Grid $x_{1:N}$ over domain
2. $K_{ij} = k(x_i, x_j)$
3. Cholesky: $K = LL^\top$
4. $\boldsymbol{f}^{(s)} = L \cdot \boldsymbol{u}$, $\boldsymbol{u} \sim \mathcal{N}(0, I)$

Five functions drawn from a GP prior with SE kernel ($\ell = 1$, $\sigma_f = 1$).



From Prior to Posterior: Bayes Rule

- **Training set:** $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$.
- **Test points:** $\boldsymbol{X}_*$ where we want predictions.
- Joint distribution of training outputs $\boldsymbol{f}$ and test outputs $\boldsymbol{f}_*$ is Gaussian.



The posterior is *still a GP* — fully characterized by updated mean and covariance.

Noiseless GP Regression

Joint distribution:

$$\begin{pmatrix} \boldsymbol{f} \\ \boldsymbol{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu} \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} K & K_* \\ K_*^\top & K_{**} \end{pmatrix} \right)$$

where $K_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$, $(K_*)_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_*^{(j)})$, $(K_{**})_{ij} = k(\boldsymbol{x}_*^{(i)}, \boldsymbol{x}_*^{(j)})$.

Posterior (conditioning on $\boldsymbol{f}$):

$$\boldsymbol{\mu}_{*|\mathcal{D}} = K_*^\top K^{-1}(\boldsymbol{f} - \boldsymbol{\mu}) + \boldsymbol{\mu}_* \tag{1}$$

$$\Sigma_{*|\mathcal{D}} = K_{**} - K_*^\top K^{-1} K_* \tag{2}$$

Key insight

- Near observed data: $K_*^\top K^{-1} K_* \approx K_{**} \Rightarrow$ small variance.
- Far from data: $K_*^\top K^{-1} K_* \approx 0 \Rightarrow$ large variance (reverts to prior).

Prediction at a Single Test Point

Noiseless setting ($\sigma_y^2 \to 0$ limit; GP interpolates training data). For a single test input $\boldsymbol{x}_*$:

$$\bar{f}_* = \boldsymbol{k}_*^\top K^{-1} \boldsymbol{f} = \sum_{i=1}^{N} \alpha_i \, k(\boldsymbol{x}_i, \boldsymbol{x}_*) \tag{3}$$

$$\sigma_*^2 = k(\boldsymbol{x}_*, \boldsymbol{x}_*) - \boldsymbol{k}_*^\top K^{-1} \boldsymbol{k}_* \tag{4}$$

where $\boldsymbol{\alpha} = K^{-1}\boldsymbol{f}$, $\boldsymbol{k}_* = (k(\boldsymbol{x}_1, \boldsymbol{x}_*), \ldots, k(\boldsymbol{x}_N, \boldsymbol{x}_*))^\top$. Noisy form ($K \to K + \sigma_y^2 I$) on next slide.

Interpretation

Posterior mean is a weighted sum of kernel basis functions centered at training points:

$$\bar{f}_* = \sum_{i=1}^{N} \alpha_i \, k(\boldsymbol{x}_i, \boldsymbol{x}_*)$$

Each observation $\boldsymbol{x}_i$ "pulls" the prediction via its kernel similarity to $\boldsymbol{x}_*$.

GP Regression with Noisy Observations

Observation model:

$$y = f(\boldsymbol{x}) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma_y^2)$$

The covariance of *observations* becomes:

$$\mathrm{cov}[y_p, y_q] = k(\boldsymbol{x}_p, \boldsymbol{x}_q) + \sigma_y^2 \delta_{pq}$$

In matrix form: replace K with

$$K_y = K + \sigma_y^2 I$$

Effect of noise

- Predictions no longer interpolate the data exactly.
- Even at training points, there is residual uncertainty.
- The noise term $\sigma_y^2 I$ also acts as numerical regularization ("nugget" / "jitter").

Noisy GPR: Posterior Equations

Posterior mean and variance with noise:

$$\bar{f}_* = \boldsymbol{k}_*^\top (K + \sigma_y^2 I)^{-1} \boldsymbol{y} \tag{5}$$

$$\sigma_*^2 = k(\boldsymbol{x}_*, \boldsymbol{x}_*) - \boldsymbol{k}_*^\top (K + \sigma_y^2 I)^{-1} \boldsymbol{k}_* \tag{6}$$

Noiseless ($\sigma_y^2 = 0$):

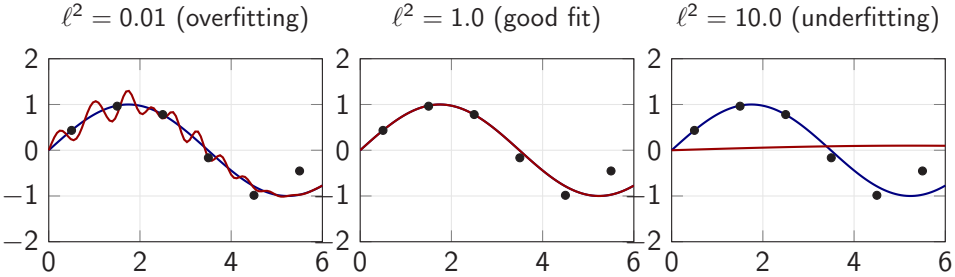
- Interpolates training points exactly
- Variance $= 0$ at training inputs
- Requires K to be positive definite

Noisy ($\sigma_y^2 > 0$):

- Smooths through training points
- Variance > 0 everywhere
- Better numerical conditioning

In practice: Even for noiseless surrogates, adding a small jitter $\sigma_y^2 \sim 10^{-6}$ improves numerical stability.

The Effect of Kernel Parameters



- **Blue:** true function $f(x) = \sin(0.9x)$.
- **Red:** GP posterior mean.
- **Too small** ℓ**:** overfits noise/wiggles. **Too large** ℓ**:** underfits, misses structure.

$\Rightarrow$ We need a principled way to learn ℓ and σ_f^2.

Learning Kernel Hyperparameters

Problem: How to choose ℓ, σ_f, σ_y? $\Rightarrow$ Maximize the marginal likelihood.

Marginal likelihood (model evidence)

$$\log p(\boldsymbol{y} \mid \boldsymbol{X}) = \underbrace{-\tfrac{1}{2}\boldsymbol{y}^\top K_y^{-1}\boldsymbol{y}}_{\text{data fit}} \underbrace{-\tfrac{1}{2}\log|K_y|}_{\text{complexity penalty}} \underbrace{-\tfrac{N}{2}\log(2\pi)}_{\text{constant}}$$

Intuition:

- **Data fit** $\uparrow$: reward explaining observations
- **Complexity** $\downarrow$: penalize overly flexible models
- Balance = automatic Occam's razor

Optimization:

- Gradient ascent on $\log p(\boldsymbol{y}|\boldsymbol{X})$
- Hyperparameters: $\theta = \{\ell, \sigma_f, \sigma_y\}$
- Closed-form gradients available
- Implemented in scikit-learn, GPyTorch

Leave-One-Out Predictive Error

Idea. Quantify out-of-sample predictive accuracy without holding out data.

For a GP with training set $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ and noise variance σ_y^2:

$$\mu_{-i}(\boldsymbol{x}_i) - y_i \;=\; -\frac{[K_y^{-1}\boldsymbol{y}]_i}{[K_y^{-1}]_{ii}}, \qquad \sigma_{-i}^2(\boldsymbol{x}_i) \;=\; \frac{1}{[K_y^{-1}]_{ii}}.$$

Why it's free.

- K_y^{-1} is recovered from the same Cholesky factor used for posterior inference
- LOO across all n points: $\mathcal{O}(n^2)$ on top of the existing $\mathcal{O}(n^3)$
- No new training, no held-out split lost

Use cases. Surrogate-health diagnostic across iterations of GP-VFI (Part IV) and across active-learning rounds in SMM (companion exercise deck).

Closed form: Rasmussen & Williams (2006), §5.4.2.

GPR: Numpy Implementation Sketch

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

def kernel(X1, X2, l=1.0, sf=1.0):
    sq = np.sum(X1**2,1).reshape(-1,1) \
         + np.sum(X2**2,1) - 2*X1@X2.T
    return sf**2 * np.exp(-0.5 * sq / l**2)

K = kernel(X_tr, X_tr) + 1e-8*np.eye(N)
L, low = cho_factor(K, lower=True)  # Cholesky
alpha = cho_solve((L, low), y_tr)   # weights

Ks = kernel(X_tr, X_te)
mu  = Ks.T @ alpha                  # mean
v   = cho_solve((L, low), Ks)
var = kernel(X_te, X_te) - Ks.T @ v # variance
```

See **Algorithm 2.1** in Rasmussen & Williams (2006), and notebook `02_GP_and_BAL.ipynb`.

GP in Practice: scikit-learn & GPyTorch

scikit-learn:

- `GaussianProcessRegressor`
- Automatic hyperparameter tuning via marginal likelihood
- Great for prototyping ($N \leq 10^4$)
- CPU only

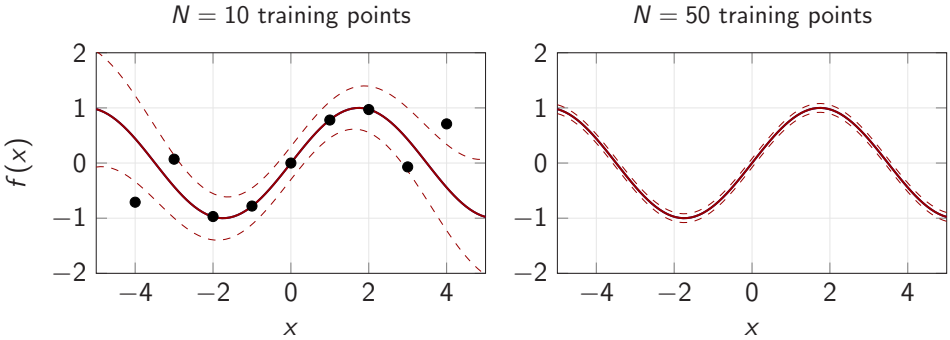
GPyTorch:

- GPU-accelerated GP inference
- Scalable to $N > 10^5$ (inducing points, KISS-GP)
- Integrates with PyTorch ecosystem
- Production-ready

Hands-on

Notebook: `02_GP_and_BAL.ipynb` — GP regression from scratch, scikit-learn, and Bayesian Active Learning.

Prediction & Confidence Intervals



Blue: true function. **Red:** posterior mean. **Dashed:** 95% confidence interval.

More data $\Rightarrow$ tighter confidence intervals.

Algorithmic Complexity

Cost of GP inference

- **Training:** Cholesky decomposition of $K_y \in \mathbb{R}^{N \times N}$: $O(N^3)$.
- **Prediction (per test point):** $O(N)$ for mean, $O(N^2)$ for variance.
- **Hyperparameter optimization:** $O(N^3)$ per gradient step.

Scaling strategies for large N

- **Sparse GPs:** Inducing point methods — approximate K with rank-M matrix ($M \ll N$), cost $O(NM^2)$.
- **Random features:** Approximate kernel with random Fourier features.
- **GPyTorch:** KISS-GP, structured kernel interpolation, GPU acceleration.

For the surrogate problems in this course ($N \leq 10^4$, $d \leq 10$), standard GPs are sufficient.

GPs vs DNNs: When to Use Which

Criterion	Gaussian Processes	Deep Neural Networks
Uncertainty quantification	Built-in (posterior variance)	Requires ensembles / MC dropout
Hyperparameter tuning	Marginal likelihood	Grid search / Bayesian opt.
Scaling with N	$O(N^3)$, limit $\sim 10^4$	$O(N)$ per epoch
Scaling with d	Curse of dimensionality	Handles $d \gg 100$
Small data ($N < 100$)	Excellent	Typically underfits
Training	Closed-form	SGD, many epochs
Interpretability	Kernel encodes prior	Black box

Rule of thumb

- $d \leq 10$, **need UQ:** GPs (+ BAL).
- $d > 10$, **large data:** DNNs.

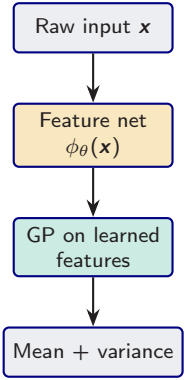
Deep Kernel Learning: Neural Features + GP Uncertainty

Idea: learn a feature map $\phi_\theta(\boldsymbol{x})$ with a neural network, then use

$$k_{\mathrm{DKL}}(\boldsymbol{x}, \boldsymbol{x}') = k_{\mathrm{base}}\big(\phi_\theta(\boldsymbol{x}), \phi_\theta(\boldsymbol{x}')\big)$$

so the GP operates in the learned feature space.

- **Front end:** the DNN learns a geometry where the target is smoother.
- **Back end:** the GP still provides posterior mean and variance.
- **Training:** optimize the feature map and kernel hyperparameters via the GP marginal likelihood.



When Do Deep Kernels Help?

Standard GP assumption: Euclidean distance in the *raw input space* is informative.

Failure modes:

- Hidden regimes / thresholds / nonstationarity
- Inputs are badly warped coordinates of a simple latent state
- Moderate-to-high-dimensional raw features with strong correlations
- You still need uncertainty quantification or BAL

Rule of thumb

- If the input is already low-dimensional and smooth: **plain GP** is usually enough.
- If the latent structure is smooth but the raw geometry is poor: **DKL** is often the right bridge.
- If d is huge and UQ is not central: **pure DNN surrogate**.

In short: DKL answers the question "Can we combine DNN feature learning with GP uncertainty?"

Summary: Gaussian Process Regression

1. **GPs** = nonparametric Bayesian regression with built-in UQ.
2. **Kernel** encodes prior beliefs (smoothness, scale, periodicity).
3. **Posterior:** closed-form mean and variance, conditioned on data.
4. **Hyperparameters:** learned by maximizing the marginal likelihood (automatic Occam's razor).
5. **Limitation:** $O(N^3)$ scaling; best for moderate N and d.
6. **DKL:** neural feature extractor + GP head = useful when raw inputs are warped or moderately high-dimensional, but you still want UQ.

Next: Use the GP's uncertainty to intelligently select training points — Bayesian Active Learning.